6. 잡

Job

- **실행된 후 종료해야 하는 성격의 작업**을 실행시킬 때 사용하는 컨트롤러
- 특정 개수만큼의 파드를 **정상적으로 실행 종료함을 보장**

1. 잡 사용하기

- 원주율을 계산하는 간단한 명령을 실행
- 잡 설정 예(job/job.yaml)

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl
        command: ["perl", "-Mbignum=bpi", "-wle", "print bp
i(2000)"]
      restartPolicy: Never
  backoffLimit: 4
```

- 잡이 사용하는 .apiVersion
- .spec.template.spec.containers[].command: 원주율 계산하는 펄 명령
- **.spec.template.spec.restartPolicy** 필드 값은 **Never**로 설정해서 파드가 항상 성공으로 끝나게 한다. Never 이에도 OnFailure도 설정할 수 있다. **OnFailure**는 파드 안 컨테이너가 비정상 종료했거나 실행이 정상 종료되지 않았을 때 컨테이너를 다시 시작하게 한다.
- .spec.backoffLimit 필드는 **잡 실행이 실패했을 때 자동으로 최대 몇 번까지 재시작할 것인지** 설정. **파드 비정상 실행 종료의 백오프정책**이라고도 한다. 기본값은 6. 보통 잡 컨트롤러가 재시작을 관리할 때마다 시간 간격을 늘린다. 처음 재시작 실패 후 **10초**를

기다린 후 시도하고 그 다음은 20초, 40초 이런 방식으로 계속 재시작 대기시간을 늘린다. 이렇게 재시작을 하다가 파드 실행이 완료되면 재시도 횟는 0으로 초기화된다.

- kubectl apply -f job.yaml
- kubectl describe job pi
- kubectl get job

```
$ kubectl describe job pi
Name:              pi
Namespace:         default
...
Completions:       1
Completion Mode:   NonIndexed
Start Time:        Sat, 04 Jan 2025 16:10:43 +0900
Completed At:      Sat, 04 Jan 2025 16:12:07 +0900
Duration:          84s
Pods Statuses:     0 Active (0 Ready) / 1 Succeeded / 0 Fail
ed
$ kubectl get job
NAME    COMPLETIONS   DURATION   AGE
pi      1/1           84s        3m30s
```

- kubectl get pods
- kubectl logs 파드명

```
$ kubectl get pods
NAME        READY   STATUS      RESTARTS   AGE
pi-4hfh9    0/1     Completed   0          4m42s
ubuntu@master:~$ kubectl logs pi-4hfh9
3.14159265358979323846264338327950288 41...
```

---

2. 잡 병렬성 관리

잡 병렬성: 잡 하나가 몇 개의 파드를 동시에 실행할 지

- .spec.parallelism 필드에서 설정

- 기본값은 1이고 0으로 설정하면 잡을 정지할 수 있다. 단 병렬성 옵션을 설정했더라도 다른 이유로 지정된 값보다 잡이 파드를 적게 실행할 수 있다.

---

3. 잡의 종류

- 단일 잡
- 완료된 잡 개수가 있는 병렬 잡
- 워크 큐가 있는 병렬 잡

---

4. 비정상적으로 실행 종료된 파드 관리하기

- 파드 안에 비정상적으로 실행 종료된 컨테이너가 있을 때를 대비해서 컨테이너 재시작 정책을 설정하는 .spec.template.spec.restartPolicy 필드를 지정할 수 있다.
  - **OnFailure**: 파드는 원래 실행 중이던 노드에서 컨테이너를 재시작한다.
  - **Never**: 재시작을 막는다.

---

5. 잡 종료와 정리

- 잡이 정상적으로 실행 종료되면 파드가 새로 생성되지도 삭제되지 않는다. 또한 잡도 남아 있다. 파드나 잡이 삭제되지 않고 남아 있으면 로그에서 에러나 경고를 확인할 수 있고, 잡의 상태도 계속해서 확인할 수 있다.
- 특정 시간을 지정해 잡 실행을 종료하려면 .spec.activeDeadlineSeconds 필드에 시간을 설정한다. 지정된 시간에 해당 잡 실행을 강제로 끝내면서 모든 파드 실행도 종료한다. 시간을 지정해 잡 실행을 끝냈다면 잡의 상태를 확인했을 때 종료 이유가 reason: DeadlineExceeded로 표시된다.
- 잡 삭제는 kubectl delete job 잡명 으로 사용자가 직접해야 한다. 잡을 삭제하면 관련된 파드들도 같이 삭제된다.

---

6. 잡 패턴

잡의 일반적인 사용 패턴

- 작업마다 잡을 하나씩 생성해 사용하는 것보다는 모든 작업을 관리하는 잡 하나를 사용하는 것이 좋다. 잡을 생성하는 오버헤드가 크기 때문이다.
- 작업 개수만큼의 파드를 생성하는 것보다 파드 하나가 여러 개의 작업을 처리하는 것이 좋다.

- 워크 큐를 사용하면 카프카나 RabbitMQ 같은 큐 서비스로 워크 큐를 구현하도록 기존 프로그램이나 컨테이너를 수정해야 한다.